DevOps Interview Notes

Docker & Kubernetes — Complete Interview Reference

Table of Contents

1. Docker — Core Concepts

- What is Docker?
- Docker Image
- Dockerfile Instructions
- Multi-stage Builds
- Build Cache
- Docker Containers
- Docker Networking
- Docker Storage
- Docker Registry
- Docker Compose
- Anti-Patterns
- Interview Deep-Dive Keywords

2. Kubernetes (K8s) — Core Concepts

- What is Kubernetes?
- Control Plane
- Pods & Workloads
- Workload Resources
- Networking
- Storage
- Configuration
- Scaling & Scheduling
- Security
- Observability (K8s)
- Deployment Strategies
- CI/CD & GitOps
- Service Mesh
- Cloud-Native Ecosystem
- Performance & Optimization
- Design Patterns
- Custom Resources & Operators
- Platform Engineering & Advanced
- Anti-Patterns
- Interview Deep-Dive Keywords

---

1. Docker — Core Concepts

What is Docker?

- **Docker** → Platform for building, shipping, and running applications in containers; "works on my machine" solved
- **Containerization** → Package app + its dependencies into a portable, isolated unit; runs consistently anywhere
- **Container Runtime** → Software executing containers; `containerd` (Docker default), `CRI-O` (K8s native), `runc` (OCI low-level)
- **Container vs VM** → Container: shares host OS kernel, MB size, ms startup; VM: full OS per instance, GB size, minutes startup; containers are lighter but less isolated
- **Namespaces** → Linux kernel feature; isolate PID, network, filesystem, user, IPC per container; what makes containers feel like VMs
- **Cgroups (Control Groups)** → Limit and account for CPU, memory, I/O, network per container; prevents one container starving others
- **Union File System** → Layered filesystem (OverlayFS); each Dockerfile instruction adds a layer; layers are read-only and shared across containers

Docker Image

- **Image** → Read-only template for creating containers; built from Dockerfile; immutable snapshot
- **Docker Image Layers** → Each `RUN`, `COPY`, `ADD` instruction creates a layer; layers are cached and reused; final image = union of all layers
- **Build Context** → Directory sent to Docker daemon during `docker build`; keep small with `.dockerignore`
- **Image Tagging** → `name:tag`; `myapp:1.2.3`; `latest` tag is a convention, not a guarantee of recency
- **Image Versioning** → Semantic versioning (`1.2.3`) + immutable SHA digest for production; never deploy `latest` in prod

Dockerfile Instructions

- **`FROM`** → Base image; first instruction; `FROM openjdk:21-jre-slim`; multi-stage uses multiple `FROM`
- **`RUN`** → Execute command at build time; creates new layer; combine with `&&` to reduce layers
- **`COPY`** → Copy files from build context to image; preferred over `ADD` for simple copies
- **`ADD`** → Like `COPY` but also supports URL download and tar auto-extraction; use `COPY` unless you need extras
- **`CMD`** → Default command when container starts; overridable at `docker run`; only last `CMD` applies
- **`ENTRYPOINT`** → Fixed command; `CMD` becomes default args; `ENTRYPOINT ["java"]` `CMD ["-jar", "app.jar"]`
- **`ENV`** → Set environment variable in image; available at runtime; `ENV JAVA_OPTS="-Xmx512m"`
- **`ARG`** → Build-time variable; NOT available at runtime; `--build-arg VERSION=1.0`

- `WORKDIR` → Set working directory; creates if missing; subsequent instructions use this as base
- `EXPOSE` → Document intended port; does NOT publish; informational only; use `-p` to actually publish

Multi-stage Builds

- **Multi-stage Build** → Multiple `FROM` in one Dockerfile; earlier stages are build environments; final stage is slim runtime image
- **Why** → Build tools (Maven, npm) not needed in final image; dramatically reduces image size; e.g. 600MB → 80MB
- **Pattern** → Stage 1: `FROM maven AS build` → compile; Stage 2: `FROM openjdk:21-jre-slim` → copy JAR only
- **Image Optimization** → Minimize layers; use slim/alpine base images; remove caches in same `RUN` layer; multi-stage

Build Cache

- **Build Cache** → Docker reuses unchanged layers; order instructions from least to most frequently changed
- **Cache Invalidation** → Any layer change invalidates all layers below; put `COPY pom.xml` before `COPY src/` to cache deps
- `.dockerignore` → Exclude files from build context; speeds up build; prevents accidentally copying secrets

Docker Containers

- **Container Lifecycle** → `create` → `start` → `running` → `stop` → `remove`; `docker run` = create + start
- **Interactive Mode** → `docker run -it`; attach stdin/stdout; for debugging; `docker exec -it <id> bash`
- **Detached Mode** → `docker run -d`; runs in background; standard for services
- **Logs** → `docker logs <id>`; `--follow` for tail; `--since 1h`; structured logs recommended
- **Exec** → `docker exec -it <container> bash`; run command in running container; for debugging
- **Restart Policies** → `no` (default), `on-failure[:n]`, `always`, `unless-stopped`; production: `unless-stopped`

Docker Networking

- **Bridge Network** → Default; containers get private IP; `docker0` bridge interface; port mapping needed for external access
- **Host Network** → Container shares host network namespace; no isolation; highest performance; `--network host`
- **Overlay Network** → Multi-host networking (Docker Swarm / Kubernetes); spans multiple Docker hosts
- **Macvlan** → Container gets its own MAC address; appears as physical device on network; for legacy apps
- **Port Mapping** → `-p host_port:container_port`; `-p 8080:80`; NAT from host to container

- **Container DNS** → Docker built-in DNS; containers resolve each other by name within same network
- **Service Discovery (Docker)** → Containers on same user-defined network reach each other by container name

Docker Storage

- **Volume** → Docker-managed storage; persists beyond container lifecycle; preferred for prod data; `docker volume create`
- **Bind Mount** → Mount host directory into container; `--mount type=bind,src=/host/path,dst=/container/path`; good for dev
- **tmpfs** → In-memory mount; not persisted; for sensitive temporary data; `--mount type=tmpfs,dst=/tmp`
- **Persistent Storage** → Use volumes for databases, logs; never store persistent data in container writable layer
- **Data Persistence** → Container writable layer is lost on remove; volumes survive container delete; always use volumes for stateful apps

Docker Registry

- **Docker Hub** → Default public registry; `docker pull nginx`; free public + paid private
- **Private Registry** → Self-hosted (`docker registry` image) or managed (AWS ECR, GCR, GHCR); for proprietary images
- **Image Pull / Push** → `docker pull image:tag`; `docker push registry/image:tag`; authentication via `docker login`
- **Image Scanning** → Scan for CVEs before deploy; Docker Scout, Trivy, Snyk; integrate in CI pipeline

Docker Compose

- **Docker Compose** → Define and run multi-container apps; `docker-compose.yml` → single command to start all services
- `docker-compose.yml` → YAML file; services (containers), networks, volumes; `version`, `services`, `networks`, `volumes` keys
- **Multi-container Apps** → `services: app: ... db: ... redis: ...`; each service is a container; compose handles networking
- **Service Definitions** → Image or build context; ports, volumes, env vars, depends_on, health checks per service
- `depends_on` → Start order; `depends_on: [db]`; does NOT wait for readiness — use health checks for that
- **Environment Variables** → `environment:` in compose; or `env_file: .env`; override with `docker-compose.override.yml`
- **Compose vs K8s** → Compose: local dev and simple single-host setups; K8s: production, multi-host, scaling, self-healing

Anti-Patterns

- **Running as root** → Security risk; use `USER nonroot` in Dockerfile
- **Large Images** → Bundle everything in one stage; use multi-stage + slim base instead

- **Hardcoded Config** → Bake config into image; use ENV vars / ConfigMaps / secrets instead
- **No Resource Limits** → One container can starve others; always set `--memory` and `--cpus`
- **Using `latest` tag in prod** → Non-deterministic; pin to specific digest or semantic version
- **Secrets in Dockerfile** → `ARG PASSWORD=secret` leaked in image history; use Docker secrets or vault

Interview Deep-Dive Keywords

- "Container = process isolation via namespaces; resource limits via cgroups; filesystem via Union FS"
- "Multi-stage builds: compile in fat image, copy artifact to slim runtime image — 10x smaller"
- "Volumes outlive containers; bind mounts are for dev; tmpfs for secrets in memory"
- "`ENTRYPOINT` is fixed; `CMD` is default args — override CMD at runtime without changing entrypoint"
- "Build cache: order instructions stable→volatile; `COPY pom.xml` before `COPY src/` caches dependency download"
- Layer caching: each instruction is a cache key; if a layer changes, all subsequent layers are rebuilt

---

2. Kubernetes (K8s) — Core Concepts

What is Kubernetes?

- **Kubernetes** → Container orchestration platform; automates deployment, scaling, healing of containerized apps; Greek for "helmsman"
- **Cluster** → Set of machines (nodes) running containerized workloads; one control plane + multiple worker nodes
- **Node** → Single machine (VM or bare-metal) in cluster; runs Pods; has `kubelet`, `kube-proxy`, container runtime
- **Pod** → Smallest deployable unit; wraps one or more containers; shared network + storage; ephemeral
- **Container vs Pod vs Deployment** → Container: app process; Pod: wrapper with IP + volumes; Deployment: manages Pod replicas + rolling updates

Control Plane

- **API Server (`kube-apiserver`)** → Front door to cluster; all operations go through it; REST API; validates + stores in etcd
- **Scheduler (`kube-scheduler`)** → Assigns Pods to Nodes; considers resource requests, affinity, taints; bin-packing algorithm
- **Controller Manager** → Runs built-in controllers (Deployment, ReplicaSet, Node); reconciliation loops; desired → actual state
- **etcd** → Distributed key-value store; source of truth for all cluster state; highly consistent (Raft consensus); back this up!

- **Reconciliation Loop** → Watch actual state → compare to desired state → take action to converge; core K8s concept
- `kubelet` → Agent on each worker node; ensures containers in Pods are running; reports node status to API server
- `kube-proxy` → Network proxy on each node; maintains iptables/IPVS rules for Service routing

Pods & Workloads

- **Pod** → One or more tightly-coupled containers; share localhost, volumes; atomic unit of scheduling; gets one cluster IP
- **Multi-container Pod** → Containers in same Pod share network/storage; communicate via `localhost`; for tightly-coupled processes
- **Init Containers** → Run sequentially before main container starts; setup tasks (DB migration, wait for dependency); must succeed
- **Sidecar Container** → Helper container alongside main; examples: log shipper (Fluentd), proxy (Envoy), vault agent
- **Ephemeral Containers** → Injected into running Pod for debugging; `kubectl debug`; no restarts; temporary
- **Static Pods** → Managed by kubelet directly (not API server); used for control plane components

Workload Resources

- **Deployment** → Manage stateless app Pods; rolling updates; rollback; `replicas`, `strategy`, `selector`; most common resource
- **ReplicaSet** → Ensures N Pod replicas running; Deployment manages ReplicaSets; rarely created directly
- **StatefulSet** → For stateful apps (databases); stable Pod names (`pod-0`, `pod-1`); stable storage per Pod; ordered deploy/scale
- **DaemonSet** → Run one Pod per node; log collectors, monitoring agents, network plugins; auto-adds to new nodes
- **Job** → Run a task to completion; `completions`, `parallelism`; e.g. batch processing, DB migration
- **CronJob** → Scheduled Jobs; cron syntax; `schedule: "0 * * * *"`; creates Job objects on schedule

Networking

- **Service** → Stable virtual IP + DNS for a set of Pods; decouples clients from ephemeral Pod IPs
- **ClusterIP** → Default; internal-only VIP; reachable within cluster; `kubectl expose deployment`
- **NodePort** → Expose service on static port on every node; `30000-32767`; external access without LB
- **LoadBalancer** → Provision cloud LB; external traffic → LB → NodePort → ClusterIP → Pods; cloud-specific
- **ExternalName** → Maps service to external DNS name; for accessing external services via K8s DNS

- **Ingress** → HTTP/HTTPS routing rules; path-based + host-based routing; SSL termination; requires Ingress Controller
- **Ingress Controller** → Implements Ingress rules; nginx-ingress, Traefik, AWS ALB Controller; must be deployed separately
- **CoreDNS** → Cluster DNS; every Service gets `<name>.<namespace>.svc.cluster.local`; Pods resolve by service name
- **Network Policies** → Firewall rules for Pods; default: all traffic allowed; `podSelector` + `ingress/egress` rules; requires CNI support
- **CNI (Container Network Interface)** → Plugin for Pod networking: Calico (network policy), Flannel (simple), Cilium (eBPF)

Storage

- **Persistent Volume (PV)** → Cluster-level storage resource; provisioned by admin or dynamically; has lifecycle independent of Pod
- **Persistent Volume Claim (PVC)** → Request for storage by user; binds to matching PV; Pod mounts PVC
- **StorageClass** → Template for dynamic PV provisioning; `provisioner` field; cloud: `aws-ebs`, `gce-pd`
- **Dynamic Provisioning** → PVC creates PV automatically via StorageClass; no manual PV pre-creation needed
- **Access Modes** → `ReadWriteOnce` (one node), `ReadOnlyMany` (many nodes), `ReadWriteMany` (many nodes; NFS/EFS)
- **StatefulSet + Storage** → Each Pod gets its own PVC (`volumeClaimTemplates`); persists across Pod restarts

Configuration

- **ConfigMap** → Store non-sensitive config as key-value pairs or files; mount as volume or inject as env vars
- **Secret** → Store sensitive data (passwords, tokens, certs); base64 encoded (NOT encrypted by default); use sealed secrets or Vault
- `envFrom` → Inject all ConfigMap/Secret keys as env vars; simpler than listing each key
- **Downward API** → Expose Pod metadata (name, namespace, labels, resource limits) to containers via env vars or volume
- `kustomize` → Kubernetes-native config customization; overlays for different environments; no templating

Scaling & Scheduling

- **HPA (Horizontal Pod Autoscaler)** → Scale Pod count based on CPU/memory/custom metrics; `targetCPUUtilizationPercentage`
- **VPA (Vertical Pod Autoscaler)** → Adjust Pod CPU/memory requests automatically; Off/Initial/Auto modes
- **Cluster Autoscaler** → Add/remove nodes based on pending Pods; cloud-provider specific; works with HPA
- **Resource Requests** → Minimum resources guaranteed; used by scheduler for placement; `requests.cpu: "250m"`
- **Resource Limits** → Maximum resources allowed; CPU throttled at limit; memory → OOMKilled if exceeded

- **Node Affinity** → Constrain Pod to nodes with specific labels; `requiredDuringScheduling` (hard) vs `preferred` (soft)
- **Pod Affinity** → Schedule Pods near (or away from) other Pods; `topologyKey: kubernetes.io/hostname`
- **Taints & Tolerations** → Taint marks node (repels Pods); Toleration on Pod allows scheduling on tainted node; for dedicated nodes
- **PodDisruptionBudget (PDB)** → Guarantee minimum available Pods during voluntary disruption (node drain, upgrades)

Security

- **RBAC (Role-Based Access Control)** → `Role`/`ClusterRole` (permissions) + `RoleBinding`/`ClusterRoleBinding` (bind to user/SA)
- **ServiceAccount** → Identity for Pods; auto-mounted token; bind RBAC roles to grant API permissions
- **Pod Security Standards** → `Privileged`, `Baseline`, `Restricted`; replace deprecated PodSecurityPolicy
- **Secrets Management** → Default K8s Secrets: base64, not encrypted; use `KMS encryption at rest` + `Sealed Secrets` or `Vault`
- **Network Policies** → Microsegmentation; default deny all + explicit allow; `podSelector` + `namespaceSelector`
- **mTLS** → Mutual TLS between services; service mesh handles automatically (Istio/Linkerd)
- `securityContext` → `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`; harden Pods

Observability (K8s)

- **Liveness Probe** → Is container healthy? Restart if fails; `httpGet`, `exec`, `tcpSocket`; don't set too aggressive
- **Readiness Probe** → Is container ready for traffic? Remove from Service endpoints if fails; separate from liveness
- **Startup Probe** → One-time check for slow-starting apps; delays liveness/readiness checks; `failureThreshold * periodSeconds`
- **Metrics Server** → Cluster-wide resource metrics (CPU/memory); enables HPA; `kubectl top pods`
- **Prometheus + Grafana** → Standard K8s monitoring stack; scrape `/metrics` from Pods; kube-state-metrics for K8s objects
- **Logging** → Logs go to stdout/stderr; aggregated by DaemonSet (Fluentd/Fluent Bit) → Elasticsearch/Loki
- **Distributed Tracing** → OpenTelemetry SDK in app + Jaeger/Zipkin collector; trace requests across services

Deployment Strategies

- **Rolling Update** → Replace Pods incrementally; `maxSurge` + `maxUnavailable`; zero-downtime; default strategy
- **Recreate** → Delete all old Pods then create new; downtime; use for incompatible schema changes

- **Blue-Green** → Full parallel environment; switch traffic via Service selector update; instant rollback; 2x resource cost
- **Canary** → Route small % to new version; increase gradually; use Ingress weight or service mesh; real traffic testing
- **Progressive Delivery** → Automated canary + analysis; Argo Rollouts; auto-promote or rollback based on metrics

CI/CD & GitOps

- **CI/CD Pipeline** → Build → Test → Push image → Update manifest → Deploy; Jenkins, GitHub Actions, GitLab CI
- **GitOps** → Git as single source of truth for cluster state; declarative; version-controlled; auditable
- **Declarative Configuration** → Define desired state in YAML; K8s reconciles; don't run `kubectl apply` manually in prod
- **Drift Detection** → GitOps tool detects when cluster state diverges from Git; alerts or auto-corrects
- **Argo CD** → Kubernetes-native GitOps; sync cluster to Git repo; UI + CLI; application-level health
- **Flux** → CNCF GitOps tool; lightweight; Helm + Kustomize support; multitenancy; image automation

Service Mesh

- **Service Mesh** → Infrastructure layer for service-to-service communication; handles retries, mTLS, tracing, circuit breaking
- **Sidecar Proxy** → Envoy proxy injected alongside each service container; intercepts all traffic in/out
- **Traffic Management** → Fine-grained routing: canary, A/B, weighted; retries, timeouts at mesh level; not in app code
- **Observability** → Golden signals automatically from mesh; distributed traces without code changes
- **Istio** → Feature-rich service mesh; Envoy sidecars; powerful but complex; `VirtualService`, `DestinationRule`, `Gateway`
- **Linkerd** → Lightweight, simpler; Rust-based proxy; lower overhead than Istio; easier operations

Cloud-Native Ecosystem

- **Cloud-native** → Designed for cloud: containers, microservices, dynamic orchestration, DevOps; CNCF defines principles
- **Multi-cluster** → Run multiple K8s clusters; fleet management; cluster federation; KubeFed, Fleet (Rancher)
- **Multi-region** → Clusters in multiple geographic regions; latency + availability; global load balancing
- **Hybrid Cloud** → On-prem + cloud K8s; consistent control plane; Anthos, EKS Anywhere, Azure Arc
- **Edge Computing** → K3s / K0s; lightweight K8s at edge; IoT devices, retail, telecoms; disconnect-tolerant

Performance & Optimization

- **Resource Requests / Limits** → Always set both; requests for scheduling, limits for protection; VPA for tuning
- **Image Size Optimization** → Multi-stage builds; distroless/slim base; remove dev tools; smaller = faster pull + less attack surface
- **Pod Scheduling Optimization** → `topologySpreadConstraints`; spread across zones; prevent single-zone SPOF
- **Node Auto-provisioning** → Karpenter (AWS); provision right-sized nodes per workload; faster + cheaper than Cluster Autoscaler
- **Limit Ranges** → Set default requests/limits per namespace; prevent unset Pods from hogging resources
- **Resource Quotas** → Cap total CPU/memory per namespace; multi-tenant clusters; prevent one team over-consuming

Design Patterns

- **Sidecar Pattern** → Co-located helper; log shipper, proxy, vault agent; shares Pod lifecycle
- **Ambassador Pattern** → Sidecar proxies outbound requests; add retry/auth/routing; client doesn't need to know service topology
- **Adapter Pattern** → Sidecar normalizes output; convert proprietary metrics format to Prometheus; standardize interfaces
- **Init Container Pattern** → Sequenced setup before main container; DB migrations, config fetching, secret init
- **Operator Pattern** → Extend K8s with custom controllers; automate Day-2 operations for stateful apps; knows app lifecycle
- **Controller Pattern** → Watch K8s resources; reconcile actual vs desired state; foundation of all K8s automation

Custom Resources & Operators

- **CRD (Custom Resource Definition)** → Extend K8s API with custom types; `KafkaCluster`, `PostgresDB`, `Certificate`
- **Custom Resource (CR)** → Instance of a CRD; managed like built-in K8s objects; `kubectl get kafkaclusters`
- **Operator** → Controller + CRD; encodes operational knowledge; e.g. postgres-operator manages backup, failover, scaling
- **Operator SDK** → Build operators with Go, Ansible, or Helm; `operator-sdk init`
- **Operator Lifecycle Manager (OLM)** → Manage operator installs, upgrades, dependencies in cluster

Platform Engineering & Advanced

- **Platform Engineering** → Build Internal Developer Platform (IDP); golden paths; self-service; reduce cognitive load
- **Internal Developer Platform (IDP)** → Abstraction over K8s; developers deploy without knowing K8s; Backstage, Humanitec
- **Multi-cluster Kubernetes** → Separate clusters per env/region/team; GitOps synchronizes all; Cluster API for provisioning

- **Control Plane vs Data Plane** → Control: API server, scheduler, etcd (make decisions); Data: kubelet, kube-proxy (execute decisions)
- **Cost Optimization (FinOps)** → Right-size Pods (VPA); spot/preemptible nodes for stateless; idle namespace cleanup; Kubecost

Anti-Patterns

- **Running Stateful Apps Incorrectly** → Deployment for stateful (no stable identity/storage); use StatefulSet
- **Large Container Images** → Slow pull, large attack surface; multi-stage + distroless + `.dockerignore`
- **Hardcoded Config** → Config in image; use ConfigMap + Secrets + environment-specific overlays
- **No Resource Limits** → Pod can consume all node resources; always set `requests` + `limits`
- **Overloaded Pods** → Multiple unrelated processes in one Pod; one process per container; separate concerns
- **Manual Deployments** → `kubectl apply` by hand; use GitOps; manual = drift, no audit trail
- **Snowflake Clusters** → Unique cluster configs not reproducible; GitOps + IaC for all cluster config

Interview Deep-Dive Keywords

- "Pod is ephemeral — never SSH into a Pod; build observability instead"
- "Deployment manages stateless; StatefulSet manages stateful — stable identity + persistent storage"
- "Service gives stable VIP; Ingress gives HTTP routing; use both: Service per app, Ingress at edge"
- "HPA scales Pods out; Cluster Autoscaler scales nodes out; VPA sizes Pod resources right"
- "etcd is the cluster brain — back it up; losing etcd = losing cluster state"
- "RBAC: principle of least privilege; every service account should have minimum permissions"
- "Rolling update: `maxSurge=1, maxUnavailable=0` ensures zero downtime but uses extra resources during update"
- GitOps: the cluster should reflect Git at all times; if cluster diverges, GitOps tool corrects it automatically
- Operator pattern: the goal is to automate everything a human operator would do — backup, failover, upgrade, scale

---

End of DevOps Interview Notes — Docker + Kubernetes